

A Hierarchical LLM-Driven Approach for Learning RDFS Ontologies from Academic Papers

No Author Given

No Institute Given

Abstract. RDFS ontologies serve as the foundation for semantic search, intelligent question answering, and knowledge graphs, providing a high-quality schema layer that defines class hierarchies, property hierarchies, and domain/range constraints. Automatically constructing them from unstructured academic papers remains challenging, especially in long-text scenarios where terminological consistency, conceptual clarity, and hierarchical property modeling are difficult to maintain. Existing methods based on rules, statistical learning, or one-step LLM generation often suffer from synonym conflicts, concept-property confusion, and weak hierarchy modeling. To address these issues, this paper proposes a hierarchical LLM-driven method for RDFS ontology learning. The process consists of three stages: a semantic network layer, a terminology normalization layer, and a schema extraction layer. A chunk-based input strategy is adopted, where local text chunks generate candidate knowledge independently before aggregation and deduplication at the paper level. Experiments on 30 academic papers from medicine, geography, and transportation show that our method generates 10,709 axioms with 95.10% sampled accuracy, compared with 90.78% for the chunked single-stage baseline and 95.47% for the non-chunked three-layer variant. The resulting ontology exhibits richer hierarchical structures, with 5,283 class nodes, 2,838 subclass edges, and a maximum depth of 6. Property-level semantic entailment increases from 8 to 707, although this increase should be interpreted cautiously because the baseline produces very few `subPropertyOf` axioms. These results indicate that chunk-based input improves knowledge coverage, while stage-wise modeling helps preserve axiom quality and structural consistency under the expanded coverage setting.

Keywords: Ontology Construction · Knowledge Graph · RDFS Ontology · Large Language Models · Prompt Engineering

1 Introduction

The Semantic Web adopts machine-processable semantic description mechanisms to ensure consistent semantic interpretation across the sharing, retrieval, and reasoning of heterogeneous resources [1]. Building upon RDF triples, RDFS (RDF Schema) furnishes fundamental modeling primitives, including classes, properties, `subClassOf` relationships, and domain/range constraints. In scenarios

involving semantic search, intelligent question answering, and knowledge graph construction, RDFS ontologies serve not only to unify terminologies but also to safeguard the stability of subsequent reasoning chains [2–4].

However, achieving these benefits with academic papers is non-trivial. Recent benchmarks [5, 6] show that even state-of-the-art large language models struggle to consistently extract and reason over structured knowledge from scholarly literature. For example, understanding formula semantics [6] or answering multi-hop questions that require integrating information across sections [5]. Scholarly articles contain a wealth of term definitions, relationship statements, and hierarchical descriptions, yet this information is often scattered across sections such as the abstract, methodology, experiments, and discussion. Without terminological consistency and structural regularity in the construction process, the resulting ontology is prone to conceptual drift, broken hierarchies, and missing property constraints. These issues directly mirror the semantic degradation and causal mismatch phenomena observed in existing retrieval-augmented systems for scientific document understanding [7]. Therefore, automatically or semi-automatically constructing RDFS ontologies from academic papers has become a task of both theoretical and practical importance. RDFS ontologies constructed from academic papers can serve as a high-quality schema layer for knowledge graphs, enabling more consistent and interpretable knowledge organization.

Text-based methods for constructing domain ontologies can generally be divided into statistically dominated and linguistically dominated approaches [8]. Statistically dominated methods exploit term frequency, co-occurrence, graph structures, and formal concept analysis to discover concepts and hierarchical relations from text [9–11]. Linguistically dominated methods rely on lexical-syntactic patterns, syntactic parsing, and semantic role labeling to identify hypernymy and semantic relations [12–14]. While these methods perform well on localized text segments, they face significant challenges when processing lengthy academic papers. Specifically, they struggle to satisfy three requirements essential for building a complete RDFS ontology: (i) cross-sectional consistency of term expressions, (ii) clear distinction between class and property roles, and (iii) standardized output covering class hierarchies, property hierarchies, domain constraints, and range constraints. In other words, the bottleneck in long-document scenarios is not merely a failure to extract, but rather a difficulty in stably organizing extracted information into a coherent and well-structured RDFS ontology.

Large Language Models (LLMs) offer a new paradigm for constructing ontologies from text. Under zero-shot or few-shot conditions, LLMs exhibit strong task generalization and chain-of-thought reasoning capabilities [15, 16], providing a general language interface for text-oriented concept identification, relation extraction, and structured knowledge generation. LLMs have been directly applied to ontology learning: Funk et al. use recursive prompting for concepts and hypernymy [17]; Li et al. employ self-evaluation and modular prompts for taxonomy [18]; Giglou et al. propose LLMs4OL for term typing and relation discovery [19]; Jamnik et al. formulate end-to-end ontology learning as learning ontology subgraphs from documents [20]; Aggarwal et al. evaluate 17 LLMs

on identifying broader/narrower/same-as relations among research topics [21]; Tang et al. introduce SLCOLM for relation extraction from classical Chinese documents [22]; Olasunkanmi et al. propose RELATE for mapping LLM-extracted relations to standardized ontology predicates [23]. These studies show LLMs can directly generate ontology structures from text, not just assist. However, most of these approaches still adopt a one-step generation paradigm, which compresses term extraction, normalization, and formal modeling into a single process. When applied to long academic papers, this practice couples multiple problems: synonymous terms are scattered across different sections, class and property roles become conflated, and although local relations can be identified, they are difficult to assemble into global hierarchical structures.

To address this gap, we propose a hierarchical LLM-based framework that decouples RDFS ontology learning from long academic papers into three stages: a semantic network layer, a terminology normalization layer, and a schema extraction layer. The semantic network layer retains raw terms and semantic relation clues from the text. The terminology normalization layer handles synonym merging, name unification, and type annotation, distinguishing classes from properties. The schema extraction layer then generates RDFS axioms, including class and property hierarchies, domain constraints, and range constraints, according to RDFS semantics. To handle information scattered across multiple sections, we adopt a chunk-based input strategy: each text chunk independently undergoes the three-stage process, and the outputs are subsequently aggregated and deduplicated at the paper level.

We evaluate our approach on 30 academic papers spanning medicine, geography, and transportation. Experimental results show that our method generates 10,709 axioms with 95.10% sampled accuracy, compared with 90.78% for a chunked single-stage baseline and 95.47% for a non-chunked three-layer variant. The resulting ontology contains 5,283 class nodes, 2,838 subclass edges, and 331 subPropertyOf edges, with a maximum depth of 6. Property-level semantic entailment increases from 8 to 707; because the baseline property hierarchy is extremely sparse, this result is interpreted as structural expansion rather than a proportional quality gain. Overall, chunk-based input and stage-wise modeling jointly enhance knowledge coverage and structural organization while maintaining high sampled axiom accuracy.

The contributions of this paper are threefold:

1. We propose a three-layer progressive framework for RDFS ontology learning from long academic texts, comprising a semantic network layer, a terminology normalization layer, and a schema extraction layer.
2. We introduce a chunk-based input and paper-level aggregation mechanism that enables the model to extract ontological knowledge from localized contexts while maintaining consistency at the paper level.
3. We construct a cross-domain evaluation dataset comprising 30 academic papers from medicine, geography, and transportation, and provide a controlled evaluation against internal baseline variants that isolate the effects of chunking and staged modeling.

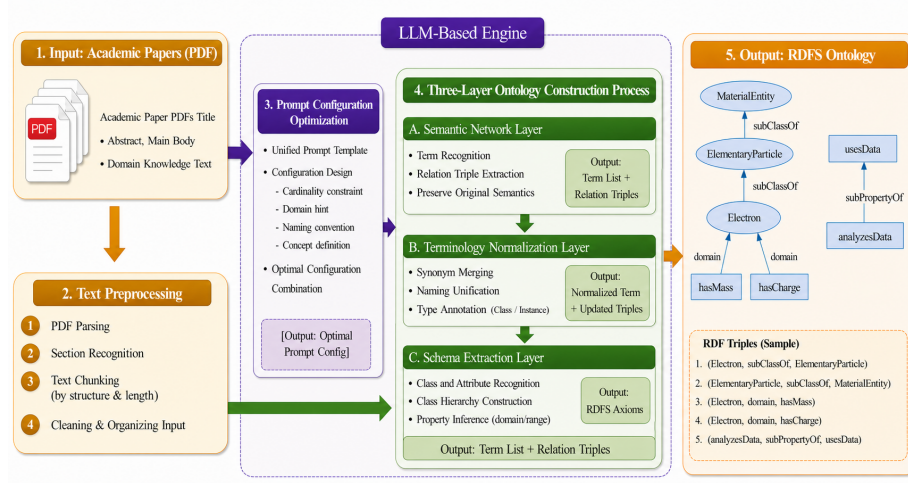


Fig. 1. The proposed hierarchy LLM-based framework to construct RDFS ontologies.

2 Methodology

In this section, we first present the overall framework of ontology construction. We then describe two core techniques that support this framework: a paragraph-based chunking algorithm and a set of prompt designs for the three layers.

2.1 Overall Framework

The overall workflow is illustrated in Figure 1, which takes unstructured academic text as input and outputs an RDFS ontology. The framework consists of three main components: text preprocessing, prompt configuration optimization, and a three-layer ontology construction process comprising a semantic network layer, a terminology normalization layer, and a schema extraction layer.

Text preprocessing: Full-length academic papers are typically long and semantically complex. If the entire text is directly inputted into an LLM, the model may omit localized concepts and relations due to context length limitations. To address this, we adopt a paragraph-based chunking input strategy: the entire paper is partitioned according to the original sequence of paragraphs, keeping the semantic integrity of each text block as intact as possible to avoid splitting closely related paragraphs across different blocks.

Prompt configuration optimization: To improve the generation quality of RDFS axioms, we design a configurable prompt scheme with multiple constraint modules and empirically determine the optimal configuration, which is then applied to the prompts used in the three layers. With the output of each layer serving as the input to the next, thereby the layers decomposing the end-to-end generation task into three subtasks: semantic extraction, term normalization, and formal modeling.

Semantic network layer: This layer takes a raw paper text as input and outputs a list of key terms and semantic relationship triples. It focuses on concepts, entities, and relationships explicitly expressed in the text, preserving complete semantic clues for downstream stages. This layer ensures semantic coverage.

Terminology normalization layer: This layer takes the output of the semantic network layer as input and yields a normalized set of terms and updated triples. This layer addresses issues such as the coexistence of synonymous terms, inconsistent naming conventions, and ambiguous term types, and annotates the semantic roles of terms within the ontology. It guarantees term consistency.

Schema extraction layer: This layer takes the normalized terms and semantic relationships as input and produces the formalized RDFS ontology structure, including class definitions, property definitions, `subClassOf` axioms, `subPropertyOf` axioms, and domain/range constraints, performing conservative modeling based on RDFS semantic rules. It enforces structural compliance.

The resulting RDFS ontology can be directly used as the schema layer of a domain-specific knowledge graph, providing formal constraints for entity typing, relation validation, and semantic reasoning.

2.2 Paragraph-based Chunking Algorithm

Algorithm 1 Bookmark-Based Hierarchical Document Chunking

Require: PDF document P , maximum chunk size M , maximum bookmark count K

Ensure: List of text chunks $\mathcal{C}$

```

1:  $bookmarks \leftarrow \text{ExtractTopLevelBookmarks}(P)$ 
2: if  $|bookmarks| < 3$  or  $|bookmarks| > K$  then
3:    $sections \leftarrow \text{ExtractFullText}(P)$ 
4: else
5:    $sections \leftarrow \text{SplitByBookmarks}(P, bookmarks)$ 
6: end if
7:  $\mathcal{C} \leftarrow \emptyset$ 
8: for all  $section \in sections$  do
9:   if  $\text{len}(section) \leq M$  then
10:     $\mathcal{C}.\text{add}(section)$ 
11:   else
12:     $\mathcal{C}.\text{add}(\text{SplitBySentences}(section, M))$ 
13:   end if
14: end for
15: return  $\mathcal{C}$ 

```

To process long scientific documents while preserving semantic boundaries, Algorithm 1 first uses PDF bookmarks as section delimiters and falls back to full-text extraction when the bookmark structure is unavailable or unreliable. Oversized sections are then split by sentence boundaries under the maximum chunk size M . The resulting chunks preserve local context for LLM extraction, and all chunk-level outputs are later aggregated and deduplicated at the paper level.

2.3 Prompt Design

All prompts share a common structure: role specification, task description, constraints, and a JSON output format. The configurable template includes quantity limitation, domain hints, naming rules, and entity definitions. These constraints are instantiated differently for the three layers. The semantic network layer extracts explicitly supported terms and triples without premature hierarchy construction; the terminology normalization layer merges synonyms, unifies names, and assigns Class/Instance/Property roles; and the schema extraction layer conservatively generates classes, properties, `subClassOf`, `subPropertyOf`, domain, and range axioms from the normalized terms and triples without introducing unsupported external concepts.

3 Experiments

3.1 Experimental Settings

All experiments use Claude Sonnet 4.6 as the primary model because it provides stable performance in long-text understanding and structured JSON generation, which are essential for producing RDFS-compliant ontology axioms. The model is invoked via API with the temperature parameter set to 0.2 to reduce randomness and improve output consistency across different text chunks. All experiments, including model inference and API calls, are implemented in Python. We compute the transitive closure using NetworkX’s `transitive_closure` function. Self-loops are excluded from the final count. Since the main experiments use a single proprietary LLM, the results should be interpreted under this model setting rather than as a model-independent conclusion.

The experiments are conducted on 30 academic papers from three domains: medicine, geography, and transportation, with 10 papers per domain. Among the three domains, transportation papers have the highest average page length (13 pages), followed by geography (11.5 pages) and medicine (8.8 pages).

3.2 Evaluation Metrics

We evaluate our method from four perspectives: efficiency, accuracy, hierarchical structure complexity, and semantic entailment.

Efficiency. Since we invoke LLMs via APIs, we use the API response time as the efficiency metric. For our three-layer method, we record the total API call time for each layer.

Accuracy. Axiom accuracy is evaluated through manual annotation. For full-paper experiments, we adopt stratified random sampling separately for each method: generated axioms are divided into four types, and 300 axioms are sampled while ensuring type coverage. Each axiom is independently annotated as correct, incorrect, or uncertain by two annotators, with disagreements resolved by a third arbitrator. Uncertain cases are excluded from the denominator. The overall accuracy is:

ID	Configuration	Number of Axioms		Accuracy
	Description	All	SC/SP/D/R	
P0	Baseline configuration	278	138/0/67/73	85.56%
P1	P0 + Quantity limitation	451	182/0/134/135	89.69%
P2	P1 + Domain hints	434	176/0/133/125	89.36%
P3	P2 + Naming rules	475	175/0/149/151	90.63%
P4	P3 + Entity definitions	468	170/0/150/148	95.70%

Table 1. Results of prompt configuration optimization (SC: subClassOf axioms, SP: subPropertyOf axioms, D: domain restrictions, and R: range restrictions.)

$$Acc = N_{correct} / (N_{sample} - N_{uncertain}) \times 100\%.$$

Hierarchical Structure Complexity. We measure class and property hierarchies separately. From the extracted subClassOf and subPropertyOf axioms, we construct directed graphs and compute four metrics: minimum depth (shortest root-to-leaf path), maximum depth (longest root-to-leaf path), average depth (average path length from reachable nodes to their roots), and the number of isolated nodes (nodes with no hierarchical relations). A deeper hierarchy and fewer isolated nodes indicate better structural organization.

Semantic Entailment. Since subClassOf and subPropertyOf are transitive in RDFS, direct axioms do not fully capture the implicit semantics of the hierarchy. We introduce semantic entailment as the total number of reachable relation pairs after computing the transitive closure of the direct hierarchy. For any relation set R , its transitive closure R^+ is defined as:

$$R^+ = \{(x, y) \mid \text{there exists a directed path of length } \geq 1 \text{ from } x \text{ to } y \text{ in } R\}.$$

Then class-level entailment $SC_c = |A_{\text{subClassOf}}^+|$ is the number of reachable ordered pairs in the transitive closure of the subClassOf graph, and property-level entailment $SC_p = |A_{\text{subPropertyOf}}^+|$ is defined analogously. Direct edges are included as reachable pairs, while isolated nodes are excluded. Thus, this metric measures potential reasoning capacity, not manually verified precision of all entailed pairs.

3.3 Preparatory Experiments

Before conducting full-text ontology construction, we perform two preparatory experiments: prompt configuration optimization and paragraph-based chunking. The former determines the optimal combination of constraint modules for RDFS axiom extraction, while the latter partitions the 30 academic papers into semantically coherent chunks for subsequent processing.

We evaluate five prompt configurations (P0–P4) on the abstracts of the 30 selected papers (see Table 1). Starting from a baseline with all constraint modules disabled (P0), we incrementally add the four modules: quantity limitation, domain hints, naming rules, and entity definitions. For each configuration, we randomly sample 100 extracted axioms from the generated set and evaluate their accuracy through manual annotation. Table 1 reports the results. A notable observation is that no subPropertyOf axioms are extracted from abstracts across all

configurations, indicating that property hierarchies are difficult to induce from summary-level text alone. Instead, `subClassOf` axioms are the most abundant category, reflecting that conceptual hierarchies are more readily captured. Among all configurations, P4 achieves the highest accuracy (95.70%) while maintaining similar axiom count (468) as P1–P3, which is significantly higher than P0 (278). The improvement from P0 to P4 also indicates that RDFS axiom quality is sensitive to prompt constraints, especially explicit entity definitions. Therefore, we adopt P4 as the prompt configuration for all three layers in the subsequent experiments.

We apply the bookmark-based hierarchical chunking algorithm described to the 30 academic papers. The medical domain papers are partitioned into an average of 9.80 chunks per paper (min: 5, max: 18), the geography domain into 12.00 chunks (min: 6, max: 29), and the transportation domain into 12.70 chunks (min: 9, max: 21). Overall, the 30 papers are divided into an average of 11.50 chunks per paper, with a minimum of 5 chunks and a maximum of 29 chunks across all papers. Among the three domains, the transportation domain exhibits the highest average number of chunks (12.70), followed by geography (12.00) and medicine (9.80). This indicates that papers in the transportation domain tend to have longer lengths or higher information density, resulting in more text chunks. The geography domain shows the largest variation in chunk counts, ranging from 6 to 29, suggesting a greater diversity in paper length within this domain. This alignment between chunk counts and original paper lengths across domains confirms that the chunking strategy preserves the structural characteristics of the data source. These chunking results provide a stable input foundation for the subsequent three-layer ontology learning process.

3.4 Results of Ontology Construction

To evaluate the effectiveness of chunking and the three-layer ontology construction method, we compare the following methods:

- **M3Layers-NoC**: This method adopts the three-layer pipeline and applies the prompt for semantic network layer directly to the original full text without chunking.
- **NoLayers**: This method applies the P4 prompt (full constraint configuration) directly to each text chunk to generate RDFS axioms, without going through the three-layer pipeline.
- **M3Layers**: This method applies the full three-layer prompts to each chunk, following the progressive pipeline of semantic network layer, terminology normalization layer, and schema extraction layer.

Overall Performance: Table 2 presents the final axioms generated after applying the three-layer pipeline and the accuracy computed over 300 sampled axioms for each method. The table shows that chunked input directly improves coverage: **NoLayers** produces 4,366 axioms, whereas **M3Layers** yields 10,709 axioms. Importantly, accuracy is reported separately for each method: **M3Layers-NoC** achieves 95.47%, **NoLayers** achieves 90.78%, and **M3Layers** achieves 95.10%.

Methods	Axiom Type	Generated Axioms	Sampled Axioms	Accuracy
M3Layers-NoC	SubClassOf	60	60	98.33%
	SubPropertyOf	4	4	100.00%
	Domain	305	134	95.28%
	Range	303	102	93.81%
	All	672	300	95.47%
NoLayers	SubClassOf	1,389	100	82.00%
	subPropertyOf	8	4	75.00%
	Domain	1,594	103	95.79%
	Range	1,375	93	96.39%
	All	4,366	300	90.78%
M3Layers	SubClassOf	2,861	82	98.73%
	subPropertyOf	340	32	82.76%
	Domain	3,812	85	91.36%
	Range	3,696	101	98.97%
	All	10,709	300	95.10%

Table 2. Performance comparison across three methods and four axiom types

Thus, the benefit of M3Layers is not merely generating more axioms, but maintaining high sampled accuracy under expanded coverage. Compared with NoLayers, it improves overall accuracy by 4.32 percentage points, suggesting that the intermediate normalization and schema extraction stages reduce cross-chunk term and role ambiguity.

Performance by Axiom Type: A more detailed breakdown by axiom type reveals distinct patterns across categories (see Table 2). For `subClassOf`, M3Layers generates 2,861 axioms (98.73% accuracy), substantially outperforming NoLayers (1,389 axioms, 82.00%). This demonstrates that the terminology normalization layer effectively resolves cross-chunk concept ambiguity. For `subPropertyOf`, M3Layers generates 340 axioms, a large increase compared to NoLayers (8 axioms) and M3Layers-NoC (4 axioms). However, its accuracy on this type (82.76%) remains lower than other types, suggesting that property hierarchy inference is still challenging and that comparisons involving this sparse baseline should be interpreted cautiously. For `Domain` constraints, all three methods achieve relatively high accuracy (91.36%–95.79%), with NoLayers slightly edging the others at 95.79%. This indicates that domain constraints are relatively easy to extract from text regardless of the pipeline design. For `Range` axioms, M3Layers achieves the highest accuracy (98.97%) while also generating the largest number of axioms (3,696), confirming that term normalization and type annotation are particularly beneficial for range constraint extraction.

Results about Hierarchical Structure: The comparative results of NoLayers and M3Layers on hierarchical structure metrics are shown in Table 3. M3Layers yields 1.63 times the number of class nodes and 2.05 times the `subClassOf` edges of NoLayers, with the proportion of isolated class nodes decreasing from 50.46% to 40.28%. For property hierarchies, M3Layers generates 331 `subPropertyOf` edges, compared with only 8 for NoLayers, and raises the maximum property depth from 1 to 6.

Evaluation Metrics	Methods	Class Hierarchy	Property Hierarchy
Number of Nodes	NoLayers	3,246	1,308
	M3Layers	5,283	1,942
Number of Edges	NoLayers	1,383	8
	M3Layers	2,838	331
Number of Isolated Nodes	NoLayers	1,638	1,298
	M3Layers	2,129	1,539
Average Path-Depth	NoLayers	0.80	0.60
	M3Layers	1.10	1.12
Maximal Path-Depth	NoLayers	5	1
	M3Layers	6	6
Semantic Entailment	NoLayers	1,992	8
	M3Layers	4,809	707

Table 3. Results about hierarchical structure and semantic entailment across two methods with chunking. Semantic entailment counts reachable pairs after transitive closure.

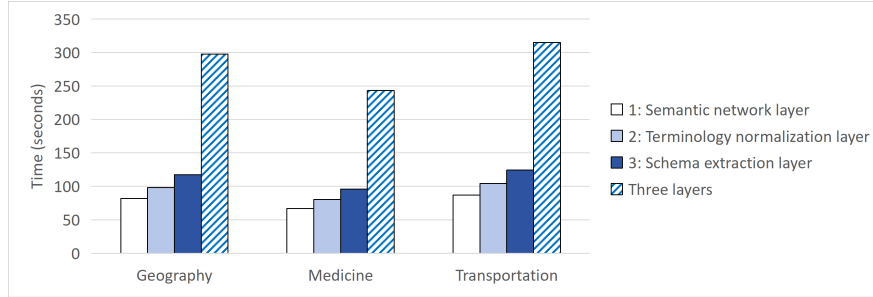


Fig. 2. Average execution time (in seconds) for invoking each layer individually and all layers combined.

Semantic Entailment: For class hierarchy, M3Layers reaches 4,809 reachable pairs, compared with 1,992 for NoLayers. The property hierarchy expands from only 8 reachable pairs to 707. Since transitive closure grows with graph connectivity and the baseline property hierarchy is very sparse, the 8-to-707 increase should be interpreted as structural expansion and potential reasoning capacity rather than independently validated precision of all entailed pairs.

Results about Efficiency: Figure 2 reveals a consistent monotonic increase in execution time from the Semantic Network Layer to the Schema Extraction Layer across all three domains. The Schema Extraction Layer dominates the computational cost, particularly for transportation (125s) and geography (120s), reflecting its role in constructing compact hierarchical structures. Medicine consistently achieves the lowest execution time across all three layers (e.g., 95s for schema extraction vs. 125s for transportation), which correlates with its shorter average paper length (8.8 pages) and lower chunk count (9.80 per paper). These results confirm that the efficiency of our three-layer approach is closely tied to the input document’s length and structural complexity.

Discussion and limitations: The results should be interpreted from both coverage and quality perspectives. Generating more axioms alone does not guarantee better ontology quality; therefore, we report per-method and per-type sampled accuracy in addition to structural metrics. Remaining limitations include the relatively small 30-paper dataset, dependence on a single proprietary LLM, the absence of external baselines with fully aligned task definitions, and the lack of downstream validation in knowledge graph construction, semantic search, or question answering.

4 Conclusion

In this paper, we propose an LLM-driven layered approach to RDFS ontology construction. The construction process is decomposed into a semantic network layer, a terminology normalization layer, and a schema extraction layer, with task-oriented prompts designed for each stage. To handle full-length academic papers, we introduce a paragraph-based chunking strategy. Experimental results on 30 academic papers spanning medicine, geography, and transportation domains show that our method generates 10,709 axioms with 95.10% sampled accuracy, improving over the chunked single-stage baseline while preserving accuracy close to the non-chunked three-layer variant. The results indicate that combining chunked input with layered modeling improves knowledge coverage and structured expressive power in RDFS ontology construction, while property-hierarchy gains should be interpreted with caution due to the sparse baseline.

Future work will validate the framework across multiple LLMs and larger, noisier PDF corpora, integrate external domain knowledge and baselines, and release benchmark datasets and annotations for downstream evaluation.

References

1. Tim Berners-Lee, James Hendler, Ora Lassila: The semantic web. *Scientific American*, 284(5): 34-43 (2001)
2. Niel Chah: OK Google, What Is Your Ontology? Or: Exploring Freebase Classification to Understand Google’s Knowledge Graph. *CoRR* abs/1805.03885 (2018)
3. Maxime Buron, Cheikh Brahim El Vaigh, François Goasdoué: Towards Faster Reformulation-based Query Answering on RDF graphs with RDFS ontologies. *ISWC* (2021)
4. Zihao Chen, Yiru Dai: Ontology-guided and neighborhood-aware embedding for industrial knowledge graph completion. *Data Knowledge Engineering*, 164: 102597 (2026)
5. Xinbang Dai, Huikang Hu, Yongrui Chen, et al.: ELAIPBENCH: A benchmark for expert-level artificial intelligence paper understanding. *DSAA* (2026)
6. Huikang Hu, Tianhao Zhang, Xinbang Dai, et al.: FORCE: A benchmark for formula reasoning and comprehension in academic papers. *DASFAA* (2026)
7. Zafar Ali, Guilin Qi, Khan Muhammad, et al.: Paper recommendation based on heterogeneous network embedding. *Knowledge-Based Systems*, 210: 106499 (2020)

8. Feiliang Ren, Jikun Shen, Binbin Sun, et al.: A survey on domain ontology construction from text. *Chinese Journal of Computers*, 42(3): 654-675 (2019)
9. Alexander Maedche, Steffen Staab: Ontology learning for the semantic web. *IEEE Intelligent Systems*, 16(2): 72-79 (2001)
10. Xiang Hou, Soh Khim Ong, Andrew Yeh Ching Nee, et al.: GRAONTO: A graph based approach for automatic construction of domain ontology. *Expert Systems with Applications*, 38(9): 11958-11975 (2011)
11. Ruchika Jindal, K R Seeja, Sonal Jain: Construction of domain ontology utilizing formal concept analysis and social media analytics. *International Journal of Cognitive Computing in Engineering*, 1: 62-69 (2020)
12. Marti A. Hearst: Automatic acquisition of hyponyms from large text corpora. *Proceedings of the 14th Conference on Computational Linguistics*, 539-545 (1992)
13. Xing Jiang, Ah Hwee Tan: CRCTOL: A semantic-based domain ontology learning system. *Journal of the American Society for Information Science and Technology*, 61(1): 150-168 (2010)
14. Mehwish Alam, Aldo Gangemi, Valentina Presutti, et al.: Semantic role labeling for knowledge graph extraction from text. *Progress in Artificial Intelligence*, 10(3): 309-320 (2021)
15. Tom Brown, Benjamin Mann, Nick Ryder, et al.: Language models are few-shot learners. *Advances in Neural Information Processing Systems* 33, 1877-1901 (2020)
16. Jason Wei, Xuezhi Wang, Dale Schuurmans, et al.: Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems* 35 (2022)
17. Marius Funk, Sebastian Hosemann, Jean Christoph Jung, et al.: Towards ontology construction with language models. *arXiv:2309.09898* (2023)
18. Jing Li, Yanda Meng, Lu Wang, et al.: Taxonomy induction using LLMs: An enhanced framework by integrating doubly-checked mechanism and self-evaluation strategy. *JIST* (2024)
19. Hamed Babaei Giglou, Jennifer D'Souza, Sören Auer: LLMs4OL: Large language models for ontology learning. *ISWC* (2023)
20. Matej Jamnik, Albert Q. Jiang, Wenda Li, et al.: End-to-end ontology learning with large language models. *Advances in Neural Information Processing Systems* 37, 87184-87225 (2024)
21. Tanay Aggarwal, Angelo Salatino, Francesco Osborne, Enrico Motta: Large language models for scholarly ontology generation: An extensive analysis in the engineering field. *Information Processing & Management*, 63(6): 104262 (2026)
22. Xuemei Tang, Linxu Wang, Jun Wang: Language model collaboration for relation extraction from classical Chinese historical documents. *Information Processing & Management*, 63(6): 104286 (2026)
23. Olawumi Olanakanmi Olanakanmi, Matthew Satsuky, Hong Yi, Chris Bizon, Harlin Lee, Stanley Ahalt: RELATE: Relation extraction in biomedical abstracts with LLMs and ontology constraints. *arXiv* (2025)